Bayesian Machine Learning in Quantitative Finance

Chapter 12: Nested Sampling for Yield Curve Model Selection

Based on Mongwe, Mbuvha & Marwala

2025

Chapter Outline

12.1 Introduction: Why Compare Yield Curve Models? I

The yield curve — the relationship between bond yield and maturity — drives debt structuring, risk management, asset pricing and portfolio allocation. The Nelson–Siegel (1987) model and its extensions are the workhorse parametric models used by portfolio managers and central banks to summarise the whole curve with a handful of interpretable numbers.

As more and more variants of these models have appeared in the literature (Diebold–Li, Svensson, Vasicek, Hull–White, ...), a practical question becomes unavoidable: *given several candidate models that all look reasonable, which one should we trust?* This is a **model selection** problem, and it can be tackled either in the frequentist paradigm (e.g. cross-validation) or in the Bayesian paradigm, via the **Bayesian evidence** (marginal likelihood).

12.1 Introduction: Why Compare Yield Curve Models? II

Two Different Bayesian Tasks (recap from earlier chapters)

- **Parameter estimation:** given *one* model, find the plausible parameter values θ — this is what MCMC methods such as Metropolis–Hastings and Hamiltonian Monte Carlo (HMC)/NUTS (Chapter 11) are built for.
- **Model selection:** given *several* candidate models, compute the marginal likelihood (evidence) of the data under each model, and compare them. This is the focus of this chapter.

Why Not Just Use the Harmonic Mean Estimator?

You may recall from the Bayesian evidence chapter that the *harmonic mean estimator* approximates Z using only posterior samples, but it is notoriously unstable: its variance can be infinite, and a handful of low-likelihood posterior draws can dominate and ruin the estimate. Other approaches such as Bayesian quadrature and the Laplace/Gaussian approximation (MacKay, 1992) either have high variance or perform badly when the true posterior is far from Gaussian or multi-modal. This chapter introduces a much more robust and general alternative: **nested sampling**.

12.1 Introduction: Why Compare Yield Curve Models? III

What This Chapter Does

1. Defines several **subsets** of the Nelson–Siegel model (Models 1–7), obtained by switching individual factor loadings $\{\beta_0, \beta_1, \beta_2\}$ on or off.
2. Introduces the **Automatic Relevance Determination Nelson–Siegel (ARD-NS)** model, which *automatically* learns how relevant each factor loading is, rather than requiring the analyst to switch loadings on/off by hand.
3. Explains the Bayesian evidence framework and **static** and **dynamic nested sampling** as ways of computing the evidence Z (and, as a free by-product, the posterior).
4. Runs numerical experiments: (a) simulated data, where the true generating model is known, to test whether the evidence framework can *recover* the correct model; and (b) real US corporate-bond yield data, calibrated with the ARD-NS model via NUTS, to see which factor loadings matter most through time.

12.1 Introduction: Why Compare Yield Curve Models? IV

Headline Result (preview)

On simulated data generated from Model 4 (level + slope only), the evidence framework correctly identifies Model 4 as the best model. On the real US corporate-bond data, the ARD-NS importances show that the *medium-term (curvature) factor loading* β_2 is the main driver of the yield curve shape over the period studied (2019–2024).

12.2.1 Subsets of the Nelson and Siegel Model I

Intuition first. The full Nelson–Siegel curve is a weighted sum of three "shape" functions of maturity τ: a constant (flat) term, a term that decays quickly from 1 to 0 (a short-term effect), and a hump-shaped term that starts at 0, rises, then falls back to 0 (a medium-term effect). If we suspect that, on a given day, only some of these shapes are really needed to explain the curve, we can *switch off* one or more of the corresponding coefficients $(\beta_0, \beta_1, \beta_2)$ and see whether the data still supports the richer model or prefers the simpler one. That comparison is exactly what the Bayesian evidence is for.

The Nelson and Siegel (1987) Yield Curve Function (Eq. 12.1)

$$y_t(\tau) = \beta_0 + \beta_1 \left(\frac{1 - \exp(-\lambda\tau)}{\lambda\tau} \right) + \beta_2 \left(\frac{1 - \exp(-\lambda\tau)}{\lambda\tau} - \exp(-\lambda\tau) \right)$$

12.2.1 Subsets of the Nelson and Siegel Model II

Notation

- $y_t(\tau)$ is the modelled yield at time t for time-to-maturity τ.
- $\{\beta_0, \beta_1, \beta_2, \lambda\}$ are the model parameters; $\beta_0 > 0$ and $\lambda > 0$ must hold.
- β_0 has loading 1 (never decays to 0) $\Rightarrow$ **long-term (level)** factor.
- β_1 has loading $\frac{1-\exp(-\lambda\tau)}{\lambda\tau}$, which starts at 1 and decays monotonically and quickly to 0 $\Rightarrow$ **short-term (slope)** factor.
- β_2 has loading $\frac{1-\exp(-\lambda\tau)}{\lambda\tau} - \exp(-\lambda\tau)$, which starts at 0, rises, then decays back to 0 $\Rightarrow$ **medium-term (curvature)** factor.
- In the Bayesian setting, the priors are Gaussian for β_1, β_2 and LogNormal for β_0, λ (to respect the positivity constraints).

12.2.1 Subsets of the Nelson and Siegel Model III

Models 1–7: Subsets of the Nelson–Siegel Model (Eqs. 12.2–12.8)

To assess the impact of each factor loading, the chapter compares seven models obtained by setting some of $\{\beta_0, \beta_1, \beta_2\}$ to zero:

Model	Formula	β_0	β_1	β_2	Interpretation
1	$y_t(\tau) = \beta_0$	✓			level only
2	$y_t(\tau) = \beta_1 \left(\frac{1-e^{-\lambda\tau}}{\lambda\tau}\right)$		✓		slope only
3	$y_t(\tau) = \beta_2 \left(\frac{1-e^{-\lambda\tau}}{\lambda\tau} - e^{-\lambda\tau}\right)$			✓	curvature only
4	$y_t(\tau) = \beta_0 + \beta_1(\cdots)$	✓	✓		level + slope
5	$y_t(\tau) = \beta_0 + \beta_2(\cdots)$	✓		✓	level + curvature
6	$y_t(\tau) = \beta_1(\cdots) + \beta_2(\cdots)$		✓	✓	slope + curvature
7	$y_t(\tau) = \beta_0 + \beta_1(\cdots) + \beta_2(\cdots)$	✓	✓	✓	**full NS model**

Model 7 is the full Nelson–Siegel model; Models 1–6 are proper subsets, each missing one or two loadings.

12.2.1 Subsets of the Nelson and Siegel Model IV

Why This Matters for Model Selection

Models 1–3 use only a single factor loading, so they can only produce *monotone or flat* yield curves — they cannot reproduce a realistically shaped curve. Models 4–7 combine two or three loadings and can fit much richer shapes. The Bayesian evidence framework (Sect. 12.3) lets us rank all seven models quantitatively, automatically penalising models that are more complex than the data requires (Occam's razor).

Python: Nelson–Siegel Loadings and the Seven Subset Models

```python
import numpy as np

def ns_loadings(tau, lam):
    """Nelson-Siegel factor loadings for maturities tau and decay lam."""
    x1 = (1 - np.exp(-lam * tau)) / (lam * tau)    # short-term (slope) loading
    x2 = x1 - np.exp(-lam * tau)                   # medium-term (curvature) loading
    return x1, x2

def yield_curve(tau, beta0, beta1, beta2, lam):
    """Full Nelson-Siegel model (Model 7, Eq. 12.8)."""
    x1, x2 = ns_loadings(tau, lam)
    return beta0 + beta1 * x1 + beta2 * x2

# Models 1-7: switch individual loadings on/off (None means "fixed at 0")
MODEL_SPECS = {
    1: dict(beta0=True,  beta1=False, beta2=False),  # level only
    2: dict(beta0=False, beta1=True,  beta2=False),  # slope only
    3: dict(beta0=False, beta1=False, beta2=True),   # curvature only
    4: dict(beta0=True,  beta1=True,  beta2=False),  # level + slope
    5: dict(beta0=True,  beta1=False, beta2=True),   # level + curvature
    6: dict(beta0=False, beta1=True,  beta2=True),   # slope + curvature
    7: dict(beta0=True,  beta1=True,  beta2=True),   # full model
}

def predict(model_id, tau, params, lam):
    spec = MODEL_SPECS[model_id]
    x1, x2 = ns_loadings(tau, lam)
```

12.2.2 Automatic Relevance Determination Nelson and Siegel Model I

Intuition first. Instead of manually enumerating and comparing all $2^3 - 1$ possible subsets of $\{\beta_0, \beta_1, \beta_2\}$, can the model *decide for itself*, from the data, which factor loadings actually matter? This is exactly the idea of **Automatic Relevance Determination (ARD)**: give every parameter its own individual prior variance (regularisation strength) α_i, and let the data tell you, via posterior inference, how large or small each α_i should be. A large α_i says "this parameter's associated factor loading is important"; a value pushed toward the prior mean (effectively shrunk to zero) says "this loading is not needed."

ARD-NS Negative Log-Likelihood (Eq. 12.9)

$$l(\mathrm{D}|\mathbf{w} = \{\beta_0, \beta_1, \beta_3, \lambda\}) = \sum_{i}^{N} \left[y_t(\tau, \{\beta_0, \beta_1, \beta_3, \lambda\}) - y_i \right]^2$$

12.2.2 Automatic Relevance Determination Nelson and Siegel Model II

Notation

- $\mathrm{D} = \{\tau, y\}$ is the observed data (maturities and yields), N is the number of observations.
- $\mathbf{w} = \{\beta_0, \beta_1, \beta_3, \lambda\}$ (as labelled in the book) are the parameters to be estimated — note this is the full three-loading (plus λ) model, i.e. the ARD version is built on top of Model 7.
- The likelihood is Gaussian, i.e. squared-error, matching the static nested sampling setup used later in the chapter.

Target Posterior (Eq. 12.10)

$$\ln p(\mathbf{w}|\mathrm{D}) = l(\mathrm{D}|\mathbf{w}) + \ln p(\mathbf{w}|\alpha) + \ln q(\alpha)$$

where $\ln p(\mathbf{w}|\alpha)$ is the log of the prior on the parameters $\mathbf{w}$ given the hyperparameters α, and $\ln q(\alpha)$ is the distribution of the hyperparameters.

12.2.2 Automatic Relevance Determination Nelson and Siegel Model III

The ARD Prior Structure

- Each parameter in $\mathbf{w}$ is given a Gaussian prior with zero mean and its own standard deviation (regularisation parameter) α_i: this is the ARD prior.
- The larger α_i is, the more important the associated parameter (factor loading) is for explaining the data.
- Because the loss surface induced by the ARD prior is multi-modal and non-convex, the chapter jointly samples $\mathbf{w}$ *and* the hyperparameters $\boldsymbol{\alpha}$ together (rather than a two-stage Gibbs scheme), which produces more stable exploration.
- As $\mathbf{w}$ is Gaussian, the α_i's are chosen to be **log-normally distributed** with mean 0 and variance 1, which makes the joint sampling of parameters and hyperparameters mathematically convenient.
- Training uses the **No-U-Turn Sampler (NUTS)** (Hoffman & Gelman, 2011) — the same HMC-based sampler from the previous chapter — because it automatically tunes the step size and path length, making it easy to use without manual tuning.

12.2.2 Automatic Relevance Determination Nelson and Siegel Model IV

Key Contrast: Fixed Subsets vs. ARD

The subset models of Sect. 12.2.1 require the analyst to *pre-commit* to a hard on/off pattern for each loading, then compare all patterns via evidence. The ARD-NS model instead learns a *continuous*, *soft* importance α_i for each loading directly from the data in a single training run — a loading is never literally deleted, but its effective contribution can be automatically shrunk toward negligible.

Python: ARD Prior and NUTS Target for the ARD-NS Model

```python
import numpy as np

def ard_ns_log_posterior(w, alpha, tau, y_obs, sigma_lik=1.0):
    """
    w     = [beta0, beta1, beta2, lam]       -- model parameters
    alpha = [a0, a1, a2, a_lam]              -- ARD hyperparameters (std devs)
    Larger alpha_i => parameter i is more "relevant" to the fit.
    """
    beta0, beta1, beta2, lam = w
    x1 = (1 - np.exp(-lam * tau)) / (lam * tau)
    x2 = x1 - np.exp(-lam * tau)
    y_pred = beta0 + beta1 * x1 + beta2 * x2

    # squared-error log-likelihood (Eq. 12.9, negated & Gaussian-normalised)
    resid = y_obs - y_pred
    log_lik = -0.5 * np.sum(resid**2) / sigma_lik**2

    # ARD prior: w_i ~ N(0, alpha_i^2)  -- larger alpha_i = less shrinkage
    log_prior_w = -0.5 * np.sum((np.array(w) / np.array(alpha))**2
                                + np.log(2 * np.pi * np.array(alpha)**2))

    # hyperprior: alpha_i ~ LogNormal(0, 1)
    log_hyperprior = -0.5 * np.sum((np.log(alpha))**2) - np.sum(np.log(alpha))

    return log_lik + log_prior_w + log_hyperprior   # Eq. 12.10

# In practice this log-posterior (and its gradient, via autodiff) is
```

12.3 The Bayesian Evidence Framework — Recap and Notation I

Intuition first. To choose between models, Bayesian inference does not stop at estimating parameters θ for a single model M — it also asks: *how well does model M, averaged over its whole prior, explain the observed data?* That number is the evidence Z_M, and comparing Z_M across models is a principled, built-in Occam's razor: a model with a large parameter space that only fits the data well in a tiny corner of that space earns a low evidence, even if its best-fit likelihood is very high.

Bayes' Theorem for Parameters Given a Model (Eq. 12.11)

$$P(\theta|\mathrm{D}, M) = \frac{P(\mathrm{D}|\theta, M)\, P(\theta|M)}{P(\mathrm{D}|M)}$$

12.3 The Bayesian Evidence Framework — Recap and Notation II

The Evidence (Marginal Likelihood) (Eq. 12.12)

$$P(\mathrm{D}|M) = \int_{\Omega_\theta} P(\mathrm{D}|\theta, M)\, P(\theta|M)\, d\theta$$

where $P(\mathrm{D}|\theta, M)$ is the likelihood, $P(\theta|M)$ the prior, and the integral runs over the whole parameter domain Ω_θ (possibly high-dimensional). This evidence *naturally penalises unnecessarily complex models*: two models can share the same best-fit likelihood but very different evidence if one wastes prior mass on parameter regions that do not fit the data well.

12.3 The Bayesian Evidence Framework — Recap and Notation III

Shorthand Notation Used in this Chapter (Eq. 12.13)

$$P(\theta_M) = \frac{L(\theta_M)\,\pi(\theta_M)}{Z_M}$$

- $P(\theta_M) := P(\theta|\mathrm{D}, M)$ is the posterior.
- $L(\theta_M) := P(\mathrm{D}|\theta, M)$ is the likelihood.
- $\pi(\theta_M) := P(\theta|M)$ is the prior.
- $Z_M := P(\mathrm{D}|M)$ is the evidence.

12.3 The Bayesian Evidence Framework — Recap and Notation IV

The Bayes Factor (Eq. 12.14)

$$BF := \frac{Z_{M_1}}{Z_{M_2}} \cdot \frac{\pi(M_1)}{\pi(M_2)}$$

where $\pi(M_i)$ is the prior belief in model i. If all models are equally likely a priori, comparing BF reduces to simply comparing Z_{M_1} and Z_{M_2}. $BF > 1$ favours M_1; $BF < 1$ favours M_2.

The Practical Problem

Z_M is usually analytically intractable for realistic models — the integral over Ω_θ has no closed form and θ can be high-dimensional. The rest of this section introduces **nested sampling** (static and dynamic), a numerical method that computes Z_M directly, with the posterior $P(\theta_M)$ falling out as a useful by-product at no extra cost.

12.3.1 Static Nested Sampling — The Core Trick I

Intuition first. The evidence integral $Z_M = \int L(\theta_M)\pi(\theta_M)\, d\theta$ is hard because θ may live in many dimensions. Nested sampling's key idea (Skilling, 2006) is to stop thinking about *where in θ-space* we are, and instead track only *how much prior probability mass currently has likelihood above some threshold*. That single number — called the **prior volume** X — turns the multidimensional integral into an ordinary, easy 1D integral.

Re-expressing the Evidence (Eq. 12.15) and the Prior Volume X

$$Z_m = \int_{\Omega_\theta} L(\theta_M)\, \pi(\theta_M)\, d\theta$$

Divide the prior parameter space Ω_θ and order it by the likelihood $L(\theta_M)$. Let X denote the total enclosed prior mass, so that each unit of prior mass is $dX = \pi(\theta_M)\, d\theta$. The likelihood, viewed as a function of X instead of θ, is written $L(\theta_M) = L(X)$.

12.3.1 Static Nested Sampling — The Core Trick II

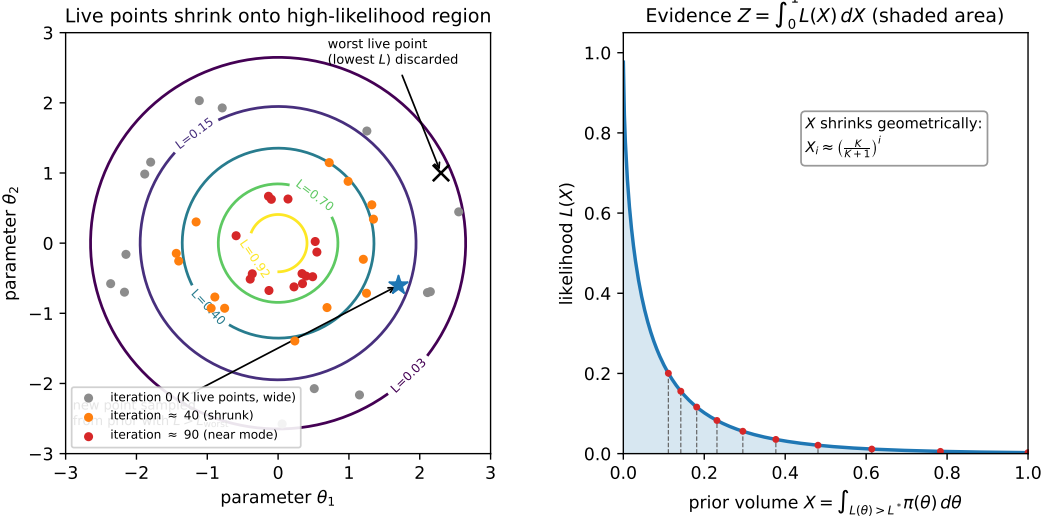
The Evidence as a 1D Integral (Eq. 12.16)

$$Z_m = \int_0^1 L(X)\, dX$$

Here $X = 1$ corresponds to the *entire* prior (no likelihood constraint yet), and $X = 0$ corresponds to the infinitesimally small region right at the peak of the likelihood. As we shrink from $X = 1$ down to $X = 0$, $L(X)$ increases monotonically (by construction, since we always discard the lowest-likelihood point) — so this really is now just the area under an ordinary, monotone 1D curve, computable with e.g. the trapezoidal rule.

12.3.1 Static Nested Sampling — The Core Trick III

Nested sampling: turning a high-dimensional integral into a 1D integral
(illustrative schematic, not from the book)



Algorithm 12.1: Static Nested Sampling (transcribed exactly)

Algorithm 12.1: Static Nested Sampling

Data: Training dataset $\{\mathbf{X}^{(i)}, \mathbf{Y}^{(i)}\}$, K live points
Result: Evidence Z_m and posterior distribution of parameters $P(\theta_M)$
begin

1. Sample K parameter values from the prior and evaluate their likelihoods.
2. Order the live points K by their likelihoods.
3. Select the live point that results in the lowest likelihood, and increment the evidence by some summation rule.
4. Remove the live point with the lowest likelihood from the original K live points.
5. Replace it with a new live point sampled from the prior within the remaining prior volume. The new live point must have a higher likelihood than the discarded live point's likelihood.
6. Repeat steps 2 to 5 until some stopping criterion is reached. This could be the desired precision on the evidence or some iteration count.

end

Reading the Algorithm

Toy Numeric Trace of Algorithm 12.1 I

To make Algorithm 12.1 completely concrete, work through a tiny toy example *by hand*. Suppose (purely for illustration) that after transforming to X-space the likelihood is known analytically as $L(X) = \exp(-X)$ for $X \in [0, 1]$, and we use $K = 5$ live points. The exact evidence is

$$Z = \int_0^1 e^{-X}\, dX = 1 - e^{-1} \approx 0.6321.$$

With $K = 5$ live points, the expected one-step shrinkage factor is $t = K/(K+1) = 5/6 \approx 0.8333$, so $X_i \approx t^{\,i}$.

Step-by-Step Trace (Steps 2–5 of Algorithm 12.1, five iterations)

Iter. i	$X_i = (5/6)^i$	$L_i = e^{-X_i}$	$\Delta X_i = X_{i-1} - X_i$	weight $\Delta X_i L_i$	running $\hat{Z}$
1	0.8333	0.4346	0.1667	0.0724	0.0724
2	0.6944	0.4994	0.1389	0.0694	0.1418
3	0.5787	0.5606	0.1157	0.0649	0.2067
4	0.4823	0.6175	0.0964	0.0595	0.2662
5	0.4019	0.6690	0.0804	0.0538	0.3200

Toy Numeric Trace of Algorithm 12.1 II

What the Numbers Show

At each iteration the worst live point (lowest likelihood) is discarded, the prior volume shrinks from X_{i-1} to X_i, and we add the rectangle of width ΔX_i and height L_i to the running evidence estimate $\hat{Z}$ (step 3 of Algorithm 12.1). After only 5 iterations we have captured 0.3200 of the true value 0.6321 — roughly half. This is expected: with only $K = 5$ live points the volume shrinks slowly, and the algorithm must keep discarding and replacing points (step 6, "repeat until some stopping criterion") for many more iterations before X becomes small enough that the remaining, unexplored volume contributes negligibly to Z. A real run would continue for on the order of $K \ln(1/X_{\text{final}})$ iterations.

Python: A From-Scratch Static Nested Sampler

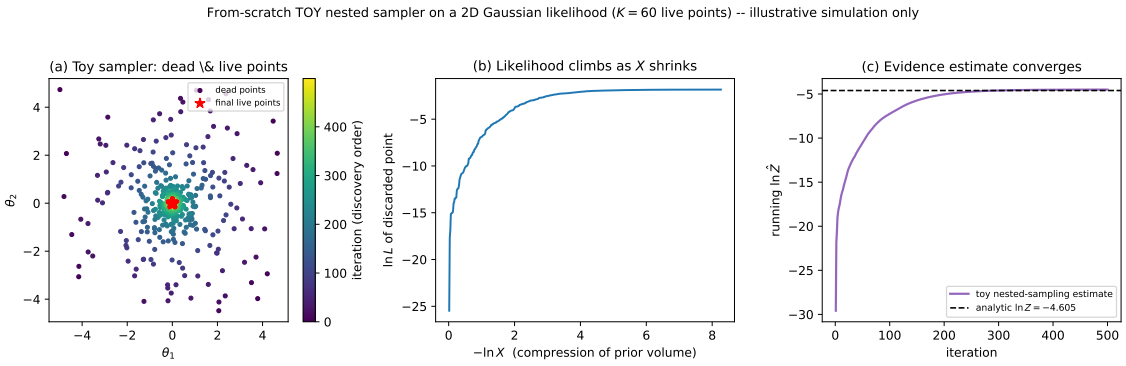
```python
import numpy as np

def static_nested_sampling(log_likelihood, bounds, n_live=50, n_iter=500):
    """
    Algorithm 12.1, steps 1-6, using rejection sampling for step 5
    (adequate for low-dimensional toy problems only).
    """
    rng = np.random.default_rng(0)
    ndim = len(bounds)
    lo = np.array([b[0] for b in bounds]); hi = np.array([b[1] for b in bounds])
    sample_prior = lambda n: lo + rng.uniform(size=(n, ndim)) * (hi - lo)

    # --- Step 1: sample K live points from the prior, evaluate likelihoods
    live_theta = sample_prior(n_live)
    live_logl = np.array([log_likelihood(t) for t in live_theta])

    logZ, logX_prev = -np.inf, 0.0              # ln Z, ln X_0 = ln(1)
    t = n_live / (n_live + 1.0)                   # expected shrink factor
    for i in range(1, n_iter + 1):
        # --- Step 2: order live points by likelihood (argmin suffices)
        # --- Step 3: select the worst, increment the evidence
        worst = np.argmin(live_logl)
        logl_worst = live_logl[worst]
        logX_i = logX_prev + np.log(t)
        dX = np.exp(logX_prev) - np.exp(logX_i)
        logZ = np.logaddexp(logZ, np.log(dX) + logl_worst)
```

From-Scratch Toy Nested Sampler in Action



The Python sampler on the previous slide run on a toy 2D Gaussian likelihood with $K = 60$ live points. **(a)** Dead points (coloured by discovery order) spiral in on the mode; the final live points (red stars) cluster tightly around it. **(b)** The likelihood of the discarded point rises steadily as $-\ln X$ (compression of the prior volume) increases. **(c)** The running evidence estimate climbs and converges to the analytic value $\ln Z \approx -4.605$ (illustrative toy simulation).

12.3.2 Dynamic Nested Sampling I

Intuition first. Static nested sampling keeps the number of live points K *fixed* for the whole run. But not every part of the run is equally important: early iterations (while X is still close to 1) mostly just explore the bulk of the prior, while the interesting action — where the likelihood rises steeply and most of the posterior mass concentrates — happens in a comparatively narrow range of X. Static nested sampling spends the same computational effort everywhere, which is wasteful. **Dynamic nested sampling** fixes this by *adaptively varying* K: it spends few live points where they add little information, and many live points exactly where the posterior mass (or the tails, if better tail resolution is needed) actually is.

12.3.2 Dynamic Nested Sampling II

How Dynamic Nested Sampling Works (Higson et al., 2019)

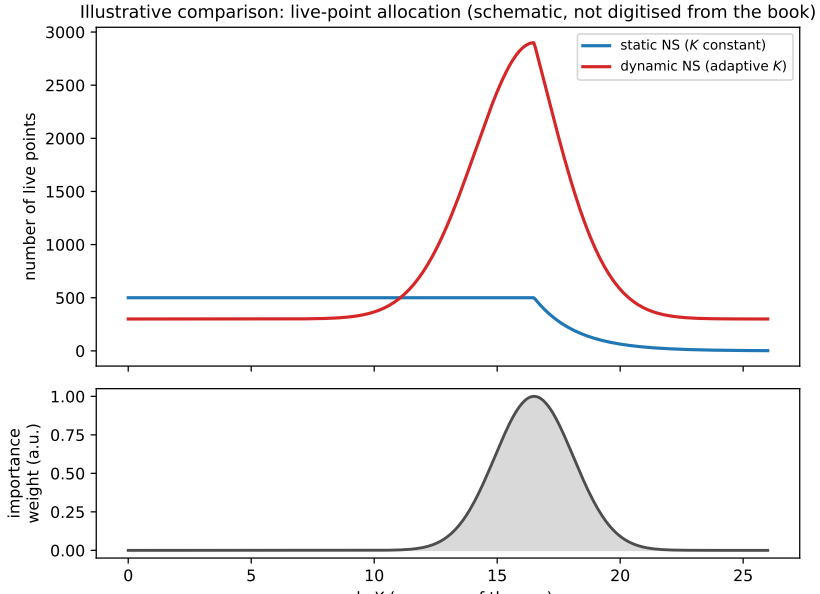
1. The distribution of posterior mass as a function of likelihood is not known ahead of time, so we first approximate it with a cheap **static** nested-sampling run using a small, constant number of live points K.
2. From this first pass, the algorithm iteratively determines the *range of likelihoods* where adding more live points will most increase calculation accuracy.
3. An additional **thread** (or batch) of live points is run specifically over that likelihood range.
4. To reduce the number of times the sampler must be restarted, several threads/batches can be created at once if needed.
5. When the batch size is small relative to the overall run, using batches larger than one has minimal effect on the efficiency gains.

12.3.2 Dynamic Nested Sampling III

Static vs. Dynamic: Key Contrast

- **Static NS:** number of live points K is *constant* throughout the run.
- **Dynamic NS:** K is *continually varied* to maximise computation accuracy for a given number of posterior samples, subject to practical limits.
- Higson et al. (2019) show empirically that dynamic nested sampling **significantly improves calculation accuracy** versus static nested sampling using the *same* number of samples, and that **both** evidence accuracy and posterior-estimation accuracy can be improved *simultaneously*.
- Unlike static nested sampling, running the dynamic algorithm for *longer* reliably yields *more accurate* results.

12.3.2 Dynamic Nested Sampling IV



Python: Static vs. Dynamic Nested Sampling with dynesty

```python
import dynesty
import numpy as np

ndim = 3  # e.g. beta0, beta1, beta2 for Model 7, lambda fixed

def loglike(theta):
    beta0, beta1, beta2 = theta
    pred = beta0 + beta1 * x1_loading + beta2 * x2_loading   # NS loadings
    resid = y_obs - pred
    return -0.5 * np.sum(resid**2) / sigma**2

def prior_transform(u):
    # map unit cube u in [0,1]^ndim to the actual prior ranges
    return -0.1 + 0.2 * u   # e.g. Uniform(-0.1, 0.1) per parameter

# --- Static nested sampling (Algorithm 12.1): K live points, fixed
sampler_static = dynesty.NestedSampler(loglike, prior_transform, ndim,
                                       nlive=500)
sampler_static.run_nested()
logZ_static = sampler_static.results.logz[-1]

# --- Dynamic nested sampling (Sect. 12.3.2): K adaptively varied
sampler_dynamic = dynesty.DynamicNestedSampler(loglike, prior_transform, ndim)
sampler_dynamic.run_nested()   # adaptively allocates live points/threads
logZ_dynamic = sampler_dynamic.results.logz[-1]

print(f"log-evidence (static)  = {logZ_static:.3f}")
```

Toy Running Example: Level+Slope vs. Level+Slope+Curvature I

This Is an Illustrative Toy Example, Not the Book's Result

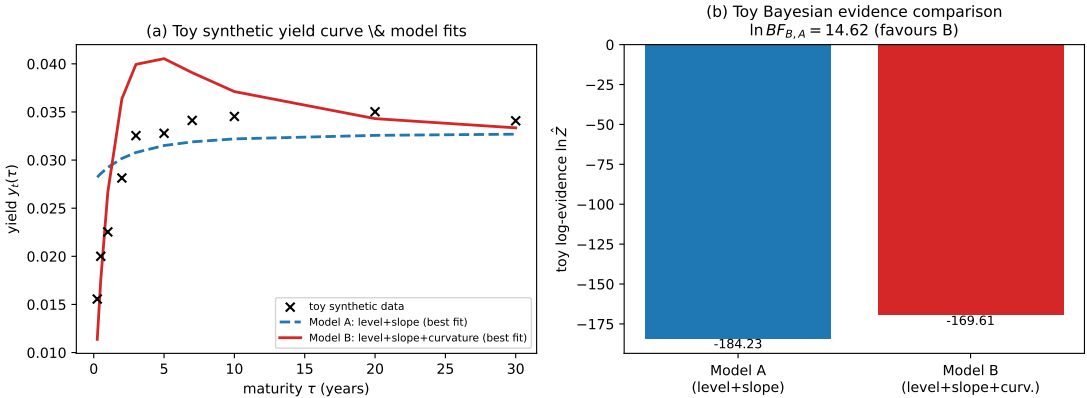
To build intuition before looking at the book's own numerical experiments (Sect. 12.4–12.5), here is a small *self-contained* example, computed from scratch with the toy nested sampler above, comparing two Nelson–Siegel-style subset models on *synthetic* data generated for this lecture only.

Toy Running Example: Level+Slope vs. Level+Slope+Curvature II

Setup

- Synthetic yield curve at $\tau \in \{0.25, 0.5, 1, 2, 3, 5, 7, 10, 20, 30\}$ years, generated from the *full* (level+slope+curvature) model with $\lambda = 0.7$, $\beta_0 = 0.035$, $\beta_1 = -0.022$, $\beta_2 = 0.018$, plus small Gaussian observation noise ($\sigma = 0.0006$).
- **Model A** (toy analogue of the book's Model 4): level + slope only, i.e. β_2 fixed at 0.
- **Model B** (toy analogue of the book's Model 7): level + slope + curvature, the full model.
- Both models share a $\mathrm{Uniform}(-0.1, 0.1)$ prior on each free β; λ is held fixed at its true value in both models to keep the toy example simple.
- Evidence for each model is computed with the from-scratch toy static nested sampler shown earlier ($K = 80$ live points).

Toy Running Example: Level+Slope vs. Level+Slope+Curvature III



Toy Running Example: Level+Slope vs. Level+Slope+Curvature IV

(a) Toy synthetic data (crosses) together with the best-fit curves of Model A (dashed blue) and Model B (solid red). Because the data was generated with genuine curvature, Model A visibly cannot track the hump in the middle of the curve. **(b)** Toy log-evidence for each model, computed by the from-scratch sampler.

Actual Numbers from This Toy Run

$$\ln \hat{Z}_A = -184.23, \qquad \ln \hat{Z}_B = -169.61, \qquad \ln BF_{B,A} = \ln \hat{Z}_B - \ln \hat{Z}_A = 14.62$$

Since $\ln BF_{B,A} \gg 0$ (equivalently $BF_{B,A} = e^{14.62} \approx 2.2 \times 10^6$), the evidence **overwhelmingly favours Model B**. This is the correct conclusion: the synthetic data really was generated with a non-zero curvature term, so the extra parameter β_2 in Model B is earning its keep — the evidence framework rewards the added complexity here *because* it is genuinely needed, rather than penalising it blindly.

12.4.1 Experiment Setup and Data Description I

Two Datasets, Two Different Purposes

- **Simulated data (for model selection):** generated from Model 4 (level + slope) with known parameters $\{\beta_0 = 0.04,\ \beta_1 = -0.05,\ \lambda = 1.3\}$. Because the true generating model is known, this lets us check whether the Bayesian evidence framework can *correctly identify* the model that produced the data.
- **Real-world data (for factor-loading analysis):** the time series of corporate bond yield curves from the US treasury/corporate market, from 1 January 2019 to 31 May 2024.

12.4.1 Experiment Setup and Data Description II

The Experimental Pipeline

1. Calibrate **all seven** subset models (Models 1–7) to the *simulated* data using **both** static and dynamic nested sampling.
2. Perform model comparison via the Bayesian evidence metric (log Z_m for each model).
3. Calibrate the **ARD-NS** model to the *real-world* data using NUTS.
4. Analyse the resulting factor-loading importances through time.

12.4.1 Experiment Setup and Data Description III

12.4.2 Performance Metrics

Predictive performance on unseen data is evaluated using:

- **MSE** (Mean Square Error)
- **MAE** (Mean Absolute Error)
- R^2 (coefficient of determination)
- **MAPE** (Mean Percentage Error)

Sampler Settings

- **NUTS (for ARD-NS):** pre-conditioning matrix $\mathbf{M} = \mathbf{I}$ (the common default in practice); 5 chains of 2000 samples each, with a burn-in of 1000 samples — sufficient for convergence on all the datasets considered.
- **Static and dynamic nested sampling:** implemented using the `dynesty` Python package (Speagle, 2020) with its **default settings**.

12.5 Results: Log Evidence Recovers the True Model I

Table 12.1: Log Evidence on Simulated Data (True Model = Model 4)

Model	Static nested sampling	Dynamic nested sampling
Model 1	-91.951 ± 0.133	-91.929 ± 0.076
Model 2	-106.615 ± 0.222	-106.559 ± 0.158
Model 3	-106.250 ± 0.219	-106.612 ± 0.152
Model 4	$\mathbf{-23.655 \pm 0.172}$	$\mathbf{-23.800 \pm 0.110}$
Model 5	-25.416 ± 0.176	-25.055 ± 0.110
Model 6	-25.512 ± 0.177	-25.402 ± 0.107
Model 7	-26.782 ± 0.192	-26.852 ± 0.119

12.5 Results: Log Evidence Recovers the True Model II

Reading Table 12.1

Static and dynamic nested sampling produce *very similar* log-evidence values for every model — reassuring agreement between the two algorithms. **Model 4 has the highest log evidence by a wide margin**, correctly identifying it as the model that generated the data. Models 1–3 (single-loading models) are the worst-ranked: they can only produce constant or monotone curves and cannot reproduce the true shape. Models 5, 6, 7 (two or three loadings, but not exactly matching the true generator) are the next-best group — they can fit the data reasonably well but are *penalised by the evidence for their unnecessary extra complexity* relative to Model 4.

12.5 Results: Log Evidence Recovers the True Model III

Predictive Performance Tells a More Nuanced Story (Tables 12.2–12.3)

On unseen simulated data, Model 4 achieves the best predictive performance under *dynamic* nested sampling calibration (e.g. $R^2 = 0.999992$, MSE $= 1.38 \times 10^{-9}$), with Model 7 a close second. Under *static* nested sampling calibration, the ranking flips slightly: Model 7 gives marginally better predictive metrics, with Model 4 second. Models 1, 2, 3 perform poorly (even negative R^2), and Model 6 is a clear outlier with very poor fit.

The Key Tension: Evidence vs. Raw Predictive Fit

Model 7 (the full model) can match Model 4's predictive accuracy almost exactly — unsurprising, since Model 7 contains Model 4 as a special case ($\beta_2 = 0$). But Model 7 has *one extra free parameter*, and the evidence metric penalises this extra flexibility unless the data actually needs it. This is Occam's razor in action: near-identical predictive performance, but the evidence still prefers the more parsimonious, correctly-specified model.

12.5 Results: Log Evidence Recovers the True Model IV

Factor Loading Importances Through Time (ARD-NS on Real Data)

Training the ARD-NS model on the real US corporate bond yield time series (2019–2024) via NUTS gives, for each day, the relative importance of the short-term (β_1), medium-term (β_2), and long-term (β_0) loadings (the λ parameter is excluded from this particular importance analysis).

Key Finding

The **medium-term (curvature) loading** β_2 is the main driver of the yield curve shape over the period studied, **followed by the short-term (slope) loading** β_1. Critically, the relative importance of each factor *varies noticeably over time* — this time-variation suggests that models which explicitly capture the temporal dynamics of the factor loadings (e.g. the dynamic Nelson–Siegel model of Diebold, Li & Yue, 2008) could be a natural complementary extension for capturing yield curve dynamics through time.

12.6 Conclusion I

What This Chapter Achieved

1. Introduced **static** and **dynamic nested sampling** as ways to compute the Bayesian evidence for yield curve model selection — a first-in-literature application to comparing subset Nelson–Siegel models.
2. Showed, on simulated data with a known generating model, that the evidence framework **correctly identifies** the true underlying model (Model 4).
3. Introduced the **Automatic Relevance Determination Nelson–Siegel (ARD-NS)** model, trained with NUTS, which **automatically ranks** which factor loadings (short-term, medium-term, long-term) are most relevant to a given yield curve.
4. On real US corporate bond data (2019–2024), found that the **medium-term (curvature)** factor loading is the dominant driver of the yield curve shape, and that this relative importance **changes through time**.

12.6 Conclusion II

Where This Framework Can Be Extended

- To other yield curve models beyond the Nelson–Siegel family.
- To other evidence-calculation methods, such as those based on normalizing flows.
- By combining with dynamic (time-varying) Nelson–Siegel formulations to explicitly capture how factor-loading importance evolves through time.

Looking Ahead

The next chapter introduces the *Bayesian investment analyst* framework, which predicts directional stock movements while automatically highlighting which input features are most relevant — a direct conceptual descendant of the automatic relevance determination idea developed here.